Day 24:故意推壞測試,確認發版被擋住

+ 補講 Day 23 加的 probe 到底是什麼、改了哪些檔案

> **一句話:**「CI 會擋壞版本」是履歷上的說法,所以要**實際推一個壞的**,看它真的被擋,而不是只在嘴上說。

正常 vs 壞測試,pipeline 走到哪

正常的 push



推了壞測試(assert 1 == 2)



└ git 裡沒有新的 release commit → ArgoCD 看不到變化 → cluster 維持舊版本

實際做了什麼

步驟	操作	觀察
1	記下目前上線版本	`d452fc30`
2	新增 `chatbot-backend/tests/test_demo_fail.py`(內容 `assert 1 == 2`),commit 並 push(`61ab3dbf`)	—
3	等 338 秒(正常 release 約 160 秒內就會出現)	**沒有** `release 61ab3dbf` commit
4	檢查 cluster	版本還是 `d452fc30`,Pod 沒被換
5	`git rm` 刪掉壞測試,push(`cc37a119`)	release 恢復,新版本自動上線,`/health` 回 200

為什麼擋得住?怎麼擋的?

機制: GitLab 的 `stages` 有先後順序。同一個 pipeline 裡,前一個 stage 只要有任何 job 失敗,後面的 stage 預設就不會跑。我們的順序是 `test → build → release`,而「改 tag 寫回 git」是在最後的 release,所以測試一紅,tag 就不會被改。

```
stages:
  - test      ← 任一 job 失敗,後面全停
  - build
  - release   ← 把新 tag 寫回 git(只有前面都綠才會到這裡)
```

這一天我做了什麼檔案變動

檔案	動作
`chatbot-backend/tests/test_demo_fail.py`	新增後又刪除(只存在於 `61ab3dbf` 到 `cc37a119` 之間)
`docs/e2e-results.md`	記錄 Day 22–24 的實測結果與已知限制

沒有改任何 CI 設定:這個「會擋」的行為是 Day 22 寫 pipeline 時就內建的,Day 24 做的是**驗證**。

誠實說明我能證明到什麼程度

我看不到 GitLab 的 pipeline 網頁。我是用**兩個間接證據**推斷的:① 等了正常時間的兩倍多,沒有出現 release commit;② cluster 上的版本沒變。這很強,但不等於親眼看到紅色的 test 階段。
你可以自己補直接證據:到 GitLab → Build → Pipelines,找 `61ab3dbf` 那一筆,截一張 test 紅、build/release 灰色(skipped)的圖放進 README。面試時拿這張圖最有說服力。

這個保護的邊界(面試加分)

它只保護「走 pipeline 的路」。如果有人**直接修改 git 裡的 `values.yaml` 把 tag 改成任意值**,ArgoCD 不會管測試,照樣部署。真正的防護還需要:main 分支設保護、強制走 Merge Request 且 pipeline 綠才能合併。這個專案沒有做,是已知限制。

補充:probe 是什麼、怎麼運作

Day 23 部署完會 502,就是少了它。這頁講清楚它怎麼做到的,以及我改了哪些檔案。

改了哪些檔案(commit `84ce3f3` 、 `032fa7d`)

檔案	加了什麼
`mychart/templates/backend-deployment.yaml`	readinessProbe、livenessProbe(探測 `/health`)、preStop
`mychart/templates/frontend-deployment.yaml`	同上(探測 `/`)

後端程式沒有改。 `/health` 這個端點 Day 1 就寫好了,probe 只是去呼叫它。

probe = K8s 定期替你「敲門問一下」

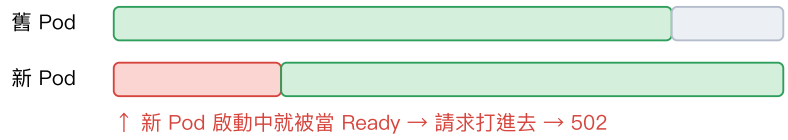



```
readinessProbe:
  httpGet: { path: /health, port: 8000 }   # 打哪裡
  periodSeconds: 3                          # 每 3 秒問一次
  failureThreshold: 3                       # 連續 3 次沒回 200 才算不及格
```

	問的問題	失敗後怎樣
readiness	準備好接客了嗎?	暫時從流量名單移掉(不殺它)
liveness	還活著嗎?	重啟容器(`initialDelaySeconds: 15` 讓它剛起來先別被問)

滾動更新時 probe 怎麼避免 502

沒有 readiness probe



有 readiness probe



preStop:另一半的保護(關閉的那一刻)

```
lifecycle:
  preStop:
    exec: { command: ["sleep", "5"] }
```

Pod 被刪除時,K8s **同時**做兩件事:通知 ingress「把它移出名單」、通知容器「開始關閉」,兩者不互相等。容器通常比名單更新快,這個空檔 ingress 還會送請求進來 → 502。preStop 讓容器先睡 5 秒才開始關,等名單更新完。

preStop 修好剩餘 502 是**我的推測**:加了之後連續 3 輪都 0 次失敗,結果一致,但我沒有抓到 ingress log 直接證明。

面試問答

Q. 怎麼證明壞版本不會上線?

> 「我實際推過一個必定失敗的測試。test 階段失敗,後面的 build 和 release 不會執行,所以沒有新 tag 寫回 git,ArgoCD 沒有東西可部署。我等了正常時間的兩倍多沒看到 release commit,cluster 版本也沒變。」

Q. 這樣就萬無一失嗎?

> 「不是。這只保護走 pipeline 的路徑。有人直接改 git 裡的 tag,ArgoCD 還是會部署。要補強需要 main 分支保護加強制 Merge Request。另外它擋得住『測試測得到的錯』,測不到的問題還是會上線,所以後續還需要部署後 smoke test 和自動回滾。」

Q. readiness 和 liveness 差在哪?

> 「readiness 問準備好接流量了嗎,失敗只是暫時不給流量;liveness 問還活著嗎,失敗會重啟容器。readiness 解決滾動更新的 502,liveness 解決卡死。」

Q. 為什麼沒 probe 會 502?

> 「K8s 預設只看容器程序有沒有跑起來,程序啟動不等於已開始監聽,中間的空檔流量就被導進去。probe 讓 K8s 真的問應用『你好了嗎』才放流量。」